CITY SENSE: Implementation Plan

One app for everything happening in the city, from fun to fixes. Mobile app (citizens + emergency crew) and Admin web dashboard. Team of 4, 14 days.

Merges three ideas: **City Sense** (agents, coordinator, civic, complaints), **Indent Error** (YOLOv8 density, DQN signals, green corridor) and **NavaBharataya** (multilingual avatar, RAG, geo-awareness).

1. Goal and success

- **Hero feature:** a multilingual city agent that answers from verified knowledge, cites its source, and abstains when it does not know.
- **Everything real:** live map + events + complaints + admin approvals, YOLOv8 density, DQN signal control (SUMO), RAG + Sarvam agent, predictive emergency routing.
- **Approach:** sequenced with fallbacks. Every feature ships a working fallback first, then the live version replaces it only when it passes its acceptance test. Nothing is cut; nothing can break the demo.
- **Resume angle:** all four members own a dedicated Spring Boot module and an end-to-end visible vertical (Platform & Governance, Multilingual Agent & RAG, Civic & Escalation, Mobility & Traffic AI).

2. Ground rules (humans and AI agents)

1. **Contracts before code.** Spring Boot is the only public API. Its OpenAPI spec generates TypeScript types for mobile and dashboard. The Python internal spec generates the Java client. Never invent an endpoint or field; change the contract first.
2. **One event model.** Every observation (CV density, complaint, flood report, agent finding) is an `Observation` in one store.
3. **Spring is the gateway.** Mobile and dashboard never call Python directly.
4. **Adapters with cached fallbacks** for every external dependency (Sarvam, LLM, SUMO, YOLO, maps). `DEMO_MODE=1` must run the full demo with the network off.
5. **High-impact actions need admin approval** (signal override, road closure, dispatch, evacuation). Low-risk notifications may auto-run. Every approval and rejection goes to the `AuditLog`.
6. **Label honestly:** `SIMULATED`, `PREDICTED`, `STALE`, `UNVERIFIED` badges in the UI. Never claim live access to real cameras, signals or dispatch.
7. Small branches, one feature each. Tests for anything with logic: routing hysteresis, escalation, RAG abstain, coordinator rules.
8. Secrets only in `.env`; keep `.env.example` current.

3. Product scope

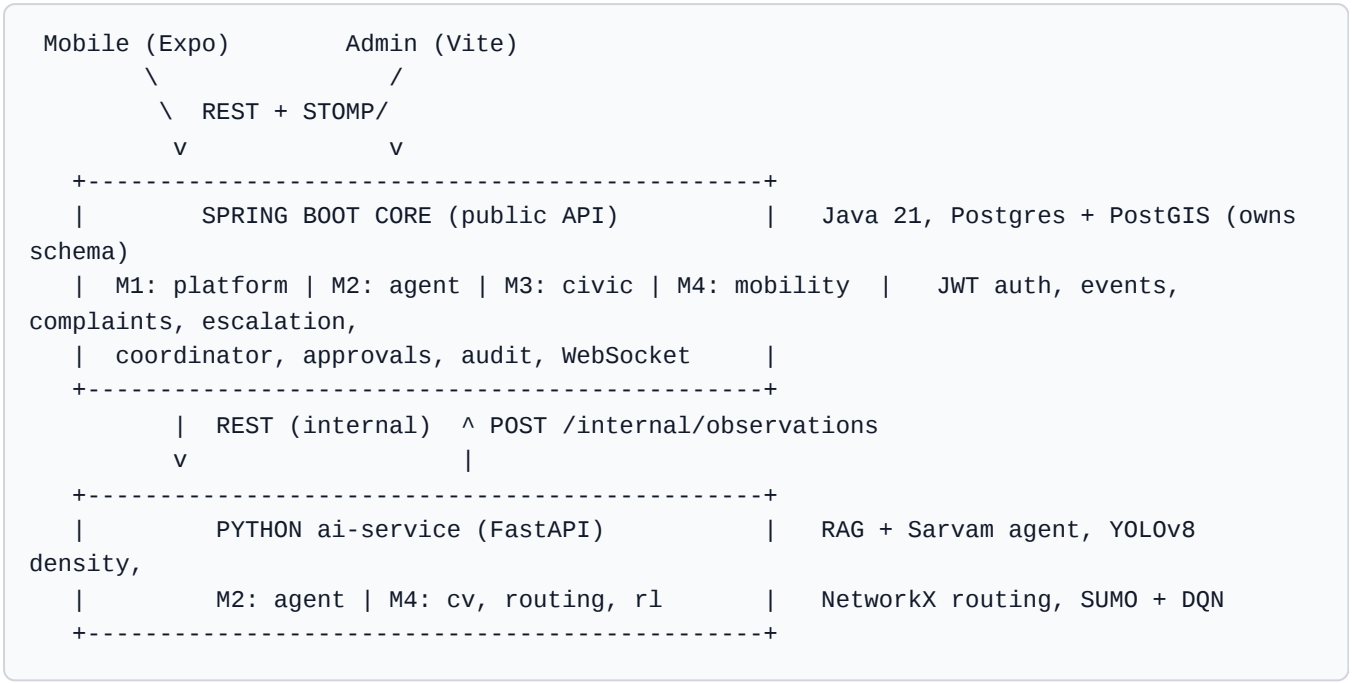
3.1 Mobile app (React Native / Expo)

Tab	What it does
Explore	Live city map and feed: events (fun) and raised issues (fixes), filters, "near me" geofence cards
Ask (hero)	City agent: text + voice, multilingual, cites sources, abstains if unsure, specialist personas (Civic, Traffic, Emergency, Transit)
Report	Raise an issue with photo, category and geo-tag; get a tracking ID
Track	Complaint timeline with the escalation ladder
Alerts	Emergency vehicle alerts ("clear the lane"), road-risk warnings; crew mode shows route and ETA

3.2 Admin web dashboard (React + Vite)

- Live map: incidents, ambulances, cameras, junctions, with the agent activity panel
- **Approval queue** and audit log
- Complaints board: assign, escalate, resolve; escalation monitor
- Events CRUD: publish city events
- Traffic panel: YOLO density per junction, signal state, RL policy vs fixed-time baseline
- Knowledge base manager: upload and verify documents for the agent
- Scenario control panel: inject flood, crash, congestion, complaint (for demos)

4. Architecture



Split of responsibility

Spring Boot (deterministic, auditable)	Python ai-service (model and simulation heavy)
Auth, users, roles	RAG retrieval and answer generation
Observations, incidents, coordinator rules	Sarvam STT / TTS / translation
Complaints and escalation scheduler	YOLOv8 grid-occupancy density
Approval queue, audit log	Route prediction, committed route, hysteresis
Feed, geofence, notifications	SUMO environment and DQN training
WebSocket broadcast, API gateway	Agent persona prompts and retrieval tools

Flow: clients call Spring. Spring calls Python for AI tasks (circuit breaker + cached fallback). Python pushes new observations (e.g. CV density) to `POST /internal/observations`. Spring stores them and broadcasts over STOMP.

5. Stack

Layer	Choice	Notes
Mobile	React Native + Expo (Expo Router), TypeScript	Shares types with dashboard

Layer	Choice	Notes
Admin	React + Vite + TypeScript, MapLibre GL, Recharts	
Core backend	**Spring Boot 3, Java 21, Maven**	Spring Web, Data JPA + Hibernate Spatial, Security (JWT), WebSocket (STOMP), springdoc-openapi, Flyway, Resilience4j, Testcontainers
AI service	Python, FastAPI	Internal only
Database	PostgreSQL + PostGIS + pgvector	Spring owns schema (Flyway); RAG tables in separate `ai` schema
Routing	NetworkX on an OSM extract of Dehradun (osmnx, cached to disk)	
CV	YOLOv8 (ultralytics) + OpenCV	Grid occupancy per lane
RL / sim	SUMO + TraCI, DQN (stable-baselines3 or custom)	
Agent	RAG on pgvector; LLM behind one `LLMClient`; Sarvam for STT/TTS/translation	Verify Sarvam quotas and language list on day 1
Deploy	Docker Compose: core, ai-service, db, dashboard; Expo Go / dev build for mobile	

6. Repository layout

```
citysense/
  contracts/              # openapi-public.yaml, openapi-internal.yaml, event.schema.json
  core/                   # Spring Boot (Java 21, Maven)
    src/main/java/com/citysense/
      platform/           # [M1: Platform Lead] auth, observations, incidents,
coordinator, audit, websocket
      agent/              # [M2: Agent Lead] gateway to ai-service, knowledge docs, chat
sessions, tool bridge
      civic/              # [M3: Civic Lead] feed, city-events, complaints, escalation,
geofence
      mobility/           # [M4: Mobility Lead] emergency routes API, actions/approvals,
traffic telemetry
      common/             # shared config, errors, adapters (fallback caches)
    src/main/resources/db/migration/   # Flyway (V1__platform, V2__agent, V3__civic,
V4__mobility)
    src/test/
  ai-service/             # FastAPI (Internal AI / ML Service)
    app/agent/            # [M2] rag.py, abstain_policy.py, personas, sarvam_client.py,
llm_client.py
    app/cv/               # [M4] yolo_density.py, grid_occupancy.py, sample_videos/
    app/routing/          # [M4] graph.py, forecast.py, commit_policy.py
    app/rl/               # [M4] sumo_env/, dqn_train.py, policy_replay.py
    tests/
  mobile/                 # React Native + Expo (Expo Router)
    app/(tabs)/
      index.tsx           # [M3] Explore (Feed & Map)
      ask.tsx             # [M2] Ask (Multilingual Hero Agent)
      report.tsx          # [M3] Report (Complaints with photo)
      track.tsx           # [M3] Track (Escalation timeline)
      alerts.tsx          # [M4] Alerts (Crew route & clear-the-lane)
  dashboard/              # React + Vite + TypeScript
    src/features/
      platform/           # [M1] live map, scenario control panel, approval queue, audit
log
      agent/              # [M2] knowledge base manager, agent activity panel
      civic/              # [M3] complaints board, events CRUD
      mobility/           # [M4] traffic panel, YOLO feed, RL policy chart
  docker-compose.yml
  .env.example
  README.md
```

7. Data model (owned by Spring, Flyway)

Entity	Key fields
`Observation`	id, type, location, observed_at, source, confidence, status, payload
`Incident`	id, type, severity, location, status, created_at, observation_ids

Entity	Key fields
`CityEvent`	id, title, category, location, starts_at, ends_at, published_by
`Complaint`	id, tracking_id, category, description, location, photo_url, status, level, warnings, assignee
`EscalationStep`	id, complaint_id, level, action, at, reason
`RoutePlan` / `RouteDecision`	plan: vehicle, edge_ids, predicted_eta, confidence, committed_at. decision: reason, old/new ETA, evidence_ids
`AgentAction`	id, incident_id, agent, action_type, payload, risk_level, approval_status
`KnowledgeDoc` / `KnowledgeChunk`	doc: title, source, verified. chunk: text, embedding (`ai` schema)
`User`	id, role (citizen / admin / crew), language
`AuditLog`	id, actor, action, timestamp, evidence, outcome

Observation types: `cv_density` , `complaint` , `flood` , `crash` , `event` , `agent_finding` .

8. Key designs

8.1 City agent (hero)

1. Input text or voice. Sarvam STT, then language detection and translation to a pivot language if needed.
2. Spring's coordinator picks a persona and forwards to `ai-service /agent/chat` .
3. Retrieve top-k verified chunks from pgvector. The LLM answers **only** from retrieved context.
4. Translate back and run TTS. The response includes `sources[]` .
5. **Abstain policy:** if similarity is below a threshold or no chunk supports the answer, say "I don't have verified information on that" and offer to file a complaint. Unit-test this.
6. **Tools the agent may request** (executed by Spring services): `get_events` , `get_complaint_status` , `create_complaint` , `get_route_eta` , `get_active_alerts` . High-impact actions are never exposed to the agent.
7. **Languages:** start with Hindi + English + 1-2 more. Check Sarvam's supported list; do not promise 22.
8. **Fallback:** text-only chat with cached answers for scripted demo questions.

8.2 Events and issues feed

One feed merges `CityEvent` , open `Complaint` and active `Incident` , filtered by category and distance. The geofence service triggers an in-app card when the user enters a radius of an event or project.

8.3 Complaints and escalation ladder

`Filed -> Worker assigned -> Warnings 1-5 (automated follow-ups) -> Ward officer -> Municipal commissioner -> CM portal`

- Spring scheduler, rule-based (no LLM). `ESCALATION_TICK_SECONDS` compresses days to seconds in demo mode.
- Each step writes `EscalationStep` and `AuditLog`, and notifies the citizen. The CM-portal step is **simulated**.

8.4 Traffic: YOLO density and RL signals

- `yolo_density.py`: sample video, YOLOv8 detections, per-lane grid occupancy score, pushed to Spring as `cv_density` every N seconds.
- SUMO scenario with 1-2 junctions. DQN state is queue length and occupancy per approach; action is phase selection; reward is negative waiting time. Compare against fixed-time and chart both.
- RL results are shown as `SIMULATED`. Do not claim the DQN controls real signals.
- **Fallback:** precomputed density JSON and a recorded SUMO run.

8.5 Emergency routing and green corridor

- Weighted graph. Edge time is estimated at the **predicted arrival time**: baseline speed x time-of-day factor x live density x incident penalty.
- **Committed route with hysteresis:** switch only if (a) the route is blocked or unsafe by verified evidence, or (b) an alternative beats the current route by a margin for a minimum dwell time. Log every decision with reason, ETAs, confidence and evidence.
- Green corridor = proposed signal windows for the route's junctions, sent to the **approval queue**. On approval: simulated signal change and citizen "clear the lane" alerts.
- **Fallback:** fixed seeded graph weights and a scripted congestion spike.

8.6 Coordinator and approvals (Spring)

Correlates observations within a time and space window into one `Incident`, ranks urgency, merges agent findings into one timeline, and creates `AgentAction` items. Rules decide; the LLM only summarises.

9. API contracts

9.1 Public (Spring Boot)

Area	Endpoints
Map and feed	`GET /api/map/features`, `GET /api/feed?lat&lng&radius&types`

Area	Endpoints
Events	`GET/POST /api/city-events`, `POST /api/events` (inject observation), `GET /api/incidents`
Complaints	`POST /api/complaints`, `GET /api/complaints/{trackingId}`, `POST /api/complaints/{id}/escalate`
Agent	`POST /api/agent/chat`, `POST /api/agent/voice`, `GET /api/agent/personas`
Emergency	`POST /api/emergency/routes`, `POST /api/emergency/reassess`
Traffic	`GET /api/traffic/junctions`, `GET /api/traffic/rl-report`
Approvals	`GET /api/actions?status=pending`, `POST /api/actions/{id}/approve`, `POST /api/actions/{id}/reject`
Admin	`POST /api/knowledge/docs`, `GET /api/audit`
Realtime	STOMP endpoint `/ws`, topic `/topic/city-events`

9.2 Internal (Python ai-service, called only by Spring)

`POST /agent/chat`, `POST /agent/voice`, `POST /cv/density`, `POST /routing/plan`, `POST /routing/reassess`, `GET /rl/report`, `POST /knowledge/ingest`. Python posts observations back to Spring at `POST /internal/observations`.

10. Team split (4 feature modules)

All four members are full-stack and each owns a dedicated **Spring Boot feature module**, their corresponding **client interface/UI panels**, and any **AI/algorithmic service** in their domain.

	Member A: Platform & Governance Lead	Member B: Multilingual Agent & RAG Lead	Member C: Civic Services & Escalation Lead	Member D: Mobility & Traffic AI Lead
Feature domain	Core Platform, Event Ingestion, Coordinator & Audit	Hero Multilingual Agent, RAG, Voice & Knowledge	Citizen Issues, Complaints Escalation & Feed	Traffic CV, Adaptive Signals & Emergency Routing

	Member A: Platform & Governance Lead	Member B: Multilingual Agent & RAG Lead	Member C: Civic Services & Escalation Lead	Member D: Mobility & Traffic AI Lead
Spring Boot module	`platform` (auth/JWT, `Observation` ingestion, `Incident` correlation engine, coordinator rules, `AgentAction` approvals, `AuditLog`, WebSocket `/ws` STOMP broadcast)	`agent` (gateway to `ai-service` `/agent/*`, knowledge doc manager `/api/knowledge/docs`, chat sessions, tool-execution bridge for complaints/events)	`civic` (city events CRUD, complaints CRUD with image upload & tracking ID, automated escalation scheduler, PostGIS geofence queries)	`mobility` (emergency routes API `/api/emergency/routes`, route reassessment, green corridor proposals, traffic junction telemetry `/api/traffic/*`)
Python / AI service	Scenario injector script (flood, crash, rain demo triggers)	`ai-service/app/agent/` (RAG on pgvector, strict abstain policy, Sarvam STT/TTS & translation, specialist personas)	Synthetic civic test fixtures & spatial seed data	`ai-service/app/cv/` (YOLOv8 lane grid density), `app/routing/` (NetworkX arrival-time routing + dwell-time hysteresis), `app/rl/` (SUMO + TraCI DQN)
Mobile app (Expo)	App shell, navigation framework, auth/profile screen, global alert banner & WebSocket listener	**Ask tab (hero feature):** multilingual voice/text chat, waveform audio recorder, source citations (`sources[]`), abstain CTA modal	**Explore, Report, Track tabs:** live map & geofenced cards, photo report form with GPS picker, interactive escalation timeline	**Alerts / Crew mode tab:** emergency crew routing view, dynamic polyline & ETA, citizen "Clear the Lane" green corridor pop-up alert
Admin dashboard (Vite)	Live operations map, Scenario control panel, Approval queue (`/api/actions`), Audit log inspector	Knowledge base manager (PDF/FAQ ingest & verification), Agent activity & confidence monitor panel	Complaints board (Kanban/table, assignment, fast-forward demo trigger), City events publisher CRUD	Traffic & Mobility panel (YOLO camera grid overlay, junction signal controls, DQN vs fixed-time comparison chart)

	Member A: Platform & Governance Lead	Member B: Multilingual Agent & RAG Lead	Member C: Civic Services & Escalation Lead	Member D: Mobility & Traffic AI Lead
Resume headline	*Spring Boot Core Platform + Distributed Event Correlation Engine + Human-in-the-Loop Governance + Real-time WebSocket STOMP*	*Enterprise RAG Architecture (pgvector) + Strict Abstain Guardrails + Multilingual Voice Agent (Sarvam AI) + Spring Resilience4j Gateway*	*Automated State Machine & Escalation Scheduler (Spring Scheduler) + Spatial PostGIS Geofencing + Citizen Civic Workflow*	*Computer Vision Traffic Telemetry (YOLOv8) + Reinforcement Learning Traffic Optimization (SUMO + DQN) + Hysteresis-Stabilized Emergency Routing*

Collaboration & shared-file rules:

1. **Module ownership:** Each member has complete ownership of their package in `core/src/main/java/com/citysense/<module>/` and their corresponding client/dashboard feature directories.
2. **Contracts & baseline:** Member A maintains the core OpenAPI contract baseline (`/contracts`) and initial DB baseline (`V1__platform.sql`). Members B, C, and D add feature-specific Flyway migrations (`V2__agent.sql` , `V3__civic.sql` , `V4__mobility.sql`) and submit contract PRs to A.
3. **Hard gate (day 4):** Member A delivers the Spring Boot spine with database, auth, and WebSocket event bus. Members B, C, and D connect their stubbed endpoints and UI components to this spine by end of day 4.

11. Timeline (14 days) with fallbacks

Days	Goal	Done when	Fallback in place
1-2	**Scaffold & contracts:** monorepo, Docker Compose (core, ai-service, db), OpenAPI contracts. Member A: Spring spine, Flyway V1, JWT stub, STOMP. Member B: RAG schema, FastAPI `/agent/chat` stub. Member C: Civic tables & CRUD stubs. Member D: OSM road graph & CV/RL stubs.	`docker compose up` starts clean; STOMP test event received by Expo & Vite clients	Stubs & mock JSON

Days	Goal	Done when	Fallback in place
2-4	**Core spine (hard gate day 4):** • A: Incident engine, scenario injector, dashboard map shell • B: Grounded RAG retrieval with citations & abstain stub, Ask tab text UI • C: Complaint filing + tracking ID, events CRUD, Report & Explore tabs • D: NetworkX shortest path with arrival time, YOLOv8 video frame loader	Flood injected -> appears on both clients over STOMP; grounded Q&A works; complaint filed & tracked	Seeded data; cached answers
5-6	**Voice, escalation & routing:** • A: Approval workflow & AuditLog persistence • B: Sarvam voice STT/TTS & Hindi/English audio round trip in Ask tab • C: Automated escalation scheduler (`ESCALATION_TICK_SECONDS`), PostGIS geofence alerts • D: Routing hysteresis (switch threshold & dwell time), mobile crew route view	Hindi voice question gets cited answer; complaint auto-escalates in demo mode; route resists minor spike	Text-only chat; scripted escalation; fixed graph weights
7-9	**Corridor, tools & simulation:** • A: Multi-observation coordinator rules (correlates rain + flood + traffic) • B: Agent tool execution (querying complaints/events via Spring) & source pill badges • C: Multi-tier escalation ladder (warnings 1-5, ward, commissioner, CM portal simulation) • D: Green corridor proposal -> approval queue -> simulated signals; SUMO + TraCI environment setup	One flood scenario yields linked findings and pending approval actions; green corridor triggers	Scripted scenario
8-11	**RL training & dashboard polish:** • A: Platform stress testing, Resilience4j circuit breakers, end-to-end audit verification • B: Knowledge base manager in dashboard, agent evaluation benchmark (20 seed Qs) • C: Dashboard complaints board (Kanban, assignment, SLA countdown) • D: DQN signal policy training vs fixed-time baseline, dashboard traffic chart & video playback	DQN vs fixed-time wait reported honestly on dashboard; full admin operations working	